

1a. Write Up -

My results:

=== Training with 100 data points ===

Fit at (1,1,1,1): 242.288936 (true = 0)

Fit at (-1,1,1,1): 124.964565 (true = 4)

Constrained (fmincon): $x = [-0.186143 \ -0.419974 \ 0.081762 \ 0.904711]$, $fval = -145.142456$

Unconstrained (fminsearch): $x = [-0.186130 \ -0.419960 \ 0.081787 \ 0.904735]$, $fval = -145.142456$

True cost at constrained point: 106.475063

True cost at unconstrained point: 106.475989

=== Training with 500 data points ===

Fit at (1,1,1,1): 114.235298 (true = 0)

Fit at (-1,1,1,1): 44.714544 (true = 4)

Constrained (fmincon): $x = [0.618506 \ 0.065453 \ -0.497034 \ -0.074635]$, $fval = -153.812819$

Unconstrained (fminsearch): $x = [0.618502 \ 0.065445 \ -0.497034 \ -0.074649]$, $fval = -153.812819$

True cost at constrained point: 48.794622

True cost at unconstrained point: 48.795720

=== Training with 1000 data points ===

Fit at (1,1,1,1): -13.547985 (true = 0)

Fit at (-1,1,1,1): -6.460310 (true = 4)

Constrained (fmincon): $x = [1.063415 \ 1.121605 \ 1.220188 \ 1.484676]$, $fval = -20.754658$

Unconstrained (fminsearch): $x = [0.933987 \ 0.829509 \ 0.558686 \ 0.197736]$, $fval = -2.996934$

True cost at constrained point: 0.220549

True cost at unconstrained point: 3.394555

1. What goes right & wrong ?

Ans. Wrong - the network can hallucinate, for me it gave -ve costs, which clearly shouldn't happen. Right - give it more data points and it does get close to actual values.

2. Difference between learned and analytical:

Ans. for the difference between analytic and learned I was thinking, that analytic has a true minima or maxima, and if you optimize it and are not constrained, that doesn't change. But with a learned, there is no true max or min, and there is a risk of hallucination, so in the process of optimizing it, especially if we are not constrained, then if the NN hallucinates an answer, the optimizer may think that is right, and continue to go in that path until you reach some nonsensical value

3. My advice :

Ans. Give more data, and constrain your model.

2.

What I did for Part 2a (linear gains):

First I tried manual tuning. I sat there for about an hour just changing the four gains in main.m and rerunning the simulation. I started with random guesses, then tried to be logical - increasing the body angle gain to react more, adding some velocity damping. I tried stuff like [5,1,50,5], [10,1,100,10], [20,5,200,20]. Every single time the robot fell within a second. I probably tried 30-40 different combinations. Figure 4 (body angle) would just spike to 1.5 and cut off. Manual tuning was useless - you can't guess four numbers that work for a nonlinear system.

Then I tried [0,0,0,0] just to see. That actually ran the full 6 seconds. Final state was [24.8, 0, 0, 0] - wheel angle huge but body angle zero. So zero gains "work" only because the robot starts perfectly upright. When I changed initial condition to [0,0,0.1,0] (small body tilt), it fell instantly. So zero gains are useless for actual balancing.

Next I did LQR. I linearized the dynamics around upright using finite differences with twip3, built A and B matrices, and used lqr with $Q = \text{diag}([1,1,100,10])$ and $R = 0.01$. Got gains around [316, 43, 316, 58] or something like that. Tested with small body tilt - it worked, kept it upright. Tested with the original initial condition (big wheel angle) - failed immediately. LQR only works near the equilibrium point.

Finally I did direct optimization with fminunc. I wrote a cost function that runs the simulation and returns negative survival time (so minimizing = maximizing survival). Started from [10,1,100,10] as initial guess. Let it run for a few hundred evaluations. The gains it found kept the robot up for a couple seconds but still eventually fell. The body angle plot looked flat but apparently crossed 1.5 at some point.

What I did for Part 2b (neural network policy):

I replaced the linear policy with a neural network: $\text{torque} = \text{network}(\text{state})$. Used MATLAB's fitnet with one hidden layer (2 neurons, tanh) and one linear output. About 13 weights total.

First I tried direct optimization with fminunc on the weight vector. Started from random weights. This was a disaster. The optimizer barely moved. The search space is 13D, cost landscape is noisy, each evaluation is slow. After hundreds of iterations, no improvement.

Then I did imitation learning. I took the LQR gains that worked for small tilts, generated 1000 random states in [-1,1], computed target torques, and trained the network with fitnet to mimic that linear controller. That worked perfectly - the network learned to reproduce the linear policy.

I took those trained weights as a starting point and ran fminunc again to fine-tune. After some iterations, I got weights that kept the robot up for about 0.123 seconds (output showed: "Robot

fell at step 41 (time 0.123 s)"). Final state was [26.6728, 12.0402, -1.5164, -11.6016]. So it fell quickly - not better than the linear controller, but at least it ran without crashing immediately.

What I learned:

Manual tuning is a waste of time for more than 2 parameters.

LQR gives you gains that work locally but fail for large disturbances.

Direct optimization with `fminunc`/`fmincon` works okay for 4 parameters but gets slow.

For neural networks, direct optimization with standard optimizers is hopeless - the parameter space is too big and the cost landscape is too noisy.

Imitation learning (cloning a working controller) is the easiest way to get a neural network policy that works, but it won't outperform the original controller.

What changes when you use a neural network for the policy representation:

More parameters: 4 gains vs 13+ weights. Optimization becomes much harder.

Need different methods: `fminunc` works for 4 parameters but fails for 13+. You need imitation learning or RL algorithms.

More expressive but harder to train: The network could theoretically represent nonlinear policies that might work better, but actually finding those weights is the hard part.

Slower evaluation: Calling the network inside the simulation loop is slightly slower than a dot product, but not the main issue.

No guarantee of improvement: My neural network just reproduced the linear controller and didn't do better. More data or better training might help, but it's not automatic.